

PROJECT PRESENTATION

# Employee Asset Management System

A role-based MERN application for tracking company assets and their assignment to employees

Built with **BMAD-METHOD**

**MERN Stack**

**JWT Auth**

**React + Tailwind CSS**

Repository: [github.com/karthick1827/employee-asset-management](https://github.com/karthick1827/employee-asset-management) | 2026-07-05

## What is BMAD-METHOD, and why it was used here

### The problem it solves

Jumping straight to code on a multi-module app (auth, employees, assets, assignments, maintenance, dashboard) causes drift — requirements get reinvented mid-build and modules stop agreeing with each other.

BMAD-METHOD splits work into a **Planning phase** (Analyst → PM → Architect → Scrum Master) that produces durable documents, and an **Implementation phase** (Dev) that builds strictly against those documents.

### How it was invoked in this project

- `npx bmad-method install` was already run in the project folder before this session, installing agent personas as Claude Code skills under `.claude/skills/`.
- The Analyst persona (**Mary**) was activated directly via the `bmad-agent-analyst` skill.
- PM, Architect, and Scrum Master stages were run in BMAD's documented **headless mode** — same templates and output contract, no back-and-forth — per the instruction to complete every step end-to-end.
- Implementation (the Dev role) was carried out directly against the resulting `architecture.md`, epic by epic.

## The four planning stages, in order

1

ANALYST · MARY  
**Product Brief**

Activated via bmad-agent-analyst.  
Captured problem, users  
(Admin/HR/Staff), scope, success  
criteria.

Output: `_bmad-output/planning-  
artifacts/briefs/.../brief.md`

2

PM · JOHN  
**PRD**

7 feature areas, 13  
numbered functional  
requirements (FR-1...FR-  
13), each with testable  
acceptance conditions.

Output:  
`.../prds/.../prd.md`  
(230 lines)

3

ARCHITECT · WINSTON  
**Architecture Spine**

5 Mongo schemas, full REST endpoint table,  
layered MVC rules (AD-1...AD-5), folder  
structure, deployment plan.

Output:  
`.../architecture/.../architecture.md`

4

SCRUM MASTER  
**Epics & Stories**

PRD + architecture  
broken into 7 epics  
and 30+  
implementation-  
ready stories with  
explicit file paths.

Output:  
`.../epics/epics-  
and-stories.md`

## WHERE & WHEN BMAD WAS USED

### Phase 5: Implementation (Dev role) — epic by epic

| Epic                                         | What was built                                                                         | Status   |
|----------------------------------------------|----------------------------------------------------------------------------------------|----------|
| Auth & Roles                                 | User model, bcrypt hashing, JWT login, role middleware, bootstrap admin seed script    | Done     |
| Employee Module                              | Employee CRUD, search/filter, soft-deactivate with active-assignment guard             | Done     |
| Asset Module                                 | Asset CRUD, status lifecycle, warranty-window queries                                  | Done     |
| Assignment Workflow                          | Assign/return endpoints, immutable history log, staff self-scoping                     | Done     |
| Maintenance Module                           | Open/resolve tickets; auto-pulls an asset from circulation and auto-returns its holder | Done     |
| Dashboard & Reports                          | Aggregate summary endpoint, CSV asset-allocation export                                | Done     |
| Frontend                                     | React/Vite SPA, Tailwind CSS, role-aware sidebar, one page per module                  | Done     |
| PDF export, notifications, SSO, barcode scan | Explicitly deferred in the PRD's MVP scope                                             | Deferred |

# Stack

## Backend

- Node.js + Express 5
- MongoDB Atlas + Mongoose 9
- JWT (jsonwebtoken) + bcrypt
- Layered MVC: routes → middleware → controllers → models

## Frontend

- React 19 + Vite
- React Router 7 (nested routes + role guards)
- Tailwind CSS 4 + lucide-react icons
- Axios with JWT-attaching interceptor

## Key architectural rule

Asset status (available / assigned / in-repair / retired) is a **single-writer field** — it can only change from inside the Assignment or Maintenance controllers, never from a direct Asset edit. This is enforced as a named invariant (AD-2) in `architecture.md`, not just a comment in code.

# Role-based access

| Capability             | Admin | HR           | Staff                    |
|------------------------|-------|--------------|--------------------------|
| Employee CRUD          | ✓     | ✓            | —                        |
| Asset CRUD             | ✓     | view only    | view only                |
| Assign / Return asset  | ✓     | —            | —                        |
| Maintenance tickets    | ✓     | —            | view own asset's tickets |
| Dashboard & CSV export | ✓     | summary only | —                        |
| My Assets view         | —     | —            | ✓ (own assignments only) |

Enforced twice: server-side via `authenticate + requireRole()` middleware on every route (the real gate), and client-side by hiding nav items a role can't use (UX only, not a security boundary).

## QUALITY

# What was actually verified

### Automated / scripted checks

- All backend files pass `node --check`
- `vite build` completes cleanly
- End-to-end curl run against a live MongoDB instance: login → create employee/asset → assign → dashboard reflects it → report broken → auto-unassigned → resolve → CSV export → staff role correctly blocked (403) on admin routes
- Live connection test against the real MongoDB Atlas cluster after credentials were corrected

### Known gap

No browser-automation tool was available in this environment, so the UI was **not** visually clicked through by the agent. The dev server was confirmed to boot and serve routes correctly, and a manual look from the actual browser is recommended before treating this as production-ready.

WHERE THIS LIVES

## Repository & running it

### GitHub

github.com/karthick1827/employee-asset-management — single initial commit, authored under the karthick1827 account. `.env` files (Mongo URI, JWT secret) are gitignored on both client and server; nothing sensitive is committed.

### Run locally

```
cd server && npm install && npm run seed && npm run dev  
cd client && npm install && npm run dev
```

Full instructions in the repo's `README.md`. Deployment plan (Render/Railway backend, Vercel/Netlify frontend, Atlas database) is documented in `architecture.md`.



# Summary

BMAD-METHOD's planning phase (Analyst → PM → Architect → Scrum Master) was used to produce a product brief, PRD, architecture spine, and epic/story breakdown *before* any code was written. The implementation phase then built every MVP epic against that architecture: auth, employee/asset/assignment/maintenance modules, dashboard, and the full React frontend — verified end-to-end against a real MongoDB Atlas database.

Deferred by design (per PRD §6.2): PDF export, notifications, SSO, barcode scanning.